

Property 1: Out-of-Coherence

Verifying that the DRAM cache is not part of the on-chip cache coherence protocol

1. What "Out of Coherence" Means

In a normal system, a CPU has multiple small caches (L1, L2, and often a shared Last-Level Cache, or LLC) sitting close to the processor. When one core modifies a piece of data, every other cache that might be holding an old copy of that same data needs to be told about it. This "keeping everyone in sync" mechanism is called **cache coherence** (protocols like MESI implement it).

A cache that is **out of coherence** is simply not part of this synchronization system. It does not send "I changed this" messages, and it does not receive "forget this, it changed" messages from anyone else. It just sits there answering a simple question — "do you have this address or not?" — with no coordination with the rest of the system.

Simple analogy: *Imagine an office where a few people share one whiteboard (this is like L1, L2, and the LLC). If someone erases something on the whiteboard, everyone in that room needs to be told, otherwise someone keeps working from outdated information — that "telling everyone" is coherence.*

Now imagine there is a photocopier sitting in the basement (this is the DRAM cache). Nobody in the office shouts down to the basement to say "erase your copy too," and the person in the basement never shouts back up either. The photocopier just answers "yes I have a copy of this page" or "no I don't" whenever someone brings a request down to it. That is exactly what "out of coherence" means for the DRAM cache — it is not part of the whiteboard-and-shouting system at all.

2. Why This Property Needs to Be True

The paper this project is based on (Babaie, Akram & Lowe-Power, ISPASS 2023) explicitly describes its DRAM cache manager as sitting "*not in the on-chip coherence domain*" and as a

"drop-in replacement for the memory controller." In other words, the design intentionally places this cache on the memory side of the system, below where coherence is handled, not inside it. If the ChampSim port secretly required coherence traffic to function correctly, it would not be a faithful implementation of the paper's architecture. This property must therefore be verified directly from the source code, not assumed.

CLAIM TO VERIFY

The DRAM cache manager (DRAM_CACHE_MANAGER) neither sends nor receives any cache-coherence messages (invalidations, snoops, or shared/exclusive state changes). It sits entirely below the on-chip coherence domain and functions purely as a memory-side device.

3. Proof — Evidence From the Actual Source Code

1 The DRAM cache is wired below the Last-Level Cache, not among the on-chip caches

The DRAM cache is connected into the memory hierarchy as the LLC's `lower_level` — the exact same wiring slot that used to hold the plain memory controller before this project's changes:

src/main.cc · line 974

```
uncore.LLC.lower_level = &uncore.DCM;
```

Everything above the LLC — L1 Instruction cache, L1 Data cache, L2 cache, and the LLC itself — forms the on-chip coherence domain. The DRAM cache sits one level below all of them, on the memory side of the system, exactly where the paper places it.

2 The only "forget this" mechanism in the whole simulator stops one level above the DRAM cache

ChampSim has exactly one function whose job is to tell a cache to erase an entry: `CACHE::invalidate_entry()`.

src/cache.cc · line 1151

```
int CACHE::invalidate_entry(uint64_t inval_addr)
{
    // finds the matching entry in this cache and marks it invalid
    ...
}
```

This function is called from exactly one place in the entire codebase — when the operating system reuses a virtual memory page, every cache that might hold stale data from that page is told to forget it, one by one:

src/main.cc · lines 533–541

```
ooo_cpu[cpu].ITLB.invalidate_entry(NRU_vpage);
ooo_cpu[cpu].DTLB.invalidate_entry(NRU_vpage);
```

```
ooo_cpu[cpu].STLB.invalidate_entry(NRU_vpage);
ooo_cpu[cpu].L1I.invalidate_entry(cl_addr);
ooo_cpu[cpu].L1D.invalidate_entry(cl_addr);
ooo_cpu[cpu].L2C.invalidate_entry(cl_addr);
uncore.LLC.invalidate_entry(cl_addr); // the list ends HERE
```

Notice this list is exhaustive — it walks through every single on-chip cache: both TLBs, the shared TLB, L1 Instruction, L1 Data, L2, and finally the LLC. There is **no eighth line** reading `uncore.DCM.invalidate_entry(cl_addr)` anywhere in the code. The DRAM cache is not an oversight that got skipped by accident — it is simply absent from the list of caches that participate in this mechanism.

3 The DRAM cache class has no coherence machinery at all — even if a message were sent, there is nowhere for it to arrive

A full text search was run across the DRAM cache manager's header and implementation files for every keyword associated with coherence:

```
grep search across inc/dram_cache_manager.h and src/dram_cache_manager.cc
```

```
grep -in "snoop\|probe\|MESI\|coherence\|invalidate" \
    inc/dram_cache_manager.h src/dram_cache_manager.cc
```

→ 0 matches in both files

The class declaration itself:

```
inc/dram_cache_manager.h · line 493
```

```
class DRAM_CACHE_MANAGER : public MEMORY {
```

DRAM_CACHE_MANAGER does not implement an `invalidate_entry()` function, does not track shared/exclusive/modified state, and has no snoop-handling code of any kind. This is not "the message gets ignored" — there is no mailbox for such a message to be delivered to in the first place.

4. Summary of Evidence

#	What was checked	Location	Result
1	Where the DRAM cache is wired into the pipeline	src/main.cc:974	Below the LLC, on the memory side
2	The one invalidation mechanism in the simulator	src/cache.cc:1151, src/main.cc:533–541	Exhaustively covers ITLB/DTLB/STLB/L1I/L1D/L2C/LLC; DRAM cache absent from the list
3	Coherence-related code inside the DRAM cache class itself	inc/dram_cache_manager.h, src/dram_cache_manager.cc	Zero matches for snoop / probe / MESI / coherence / invalidate

CONCLUSION

The absence of coherence participation is **structural**, not a coincidence of how the simulator happened to be tested. Two independent facts prove this: (a) the only invalidation mechanism that exists in ChampSim is enumerated explicitly, cache by cache, and stops one level above the DRAM cache, and (b) the DRAM cache class itself provides no method that could even receive such a message. Both the sending side and the receiving side of coherence traffic are verifiably absent for the DRAM cache manager.

This matches the reference design described in Babaie, Akram & Lowe-Power (ISPASS 2023), which explicitly states the DRAM cache manager is placed "*not in the on-chip coherence domain*" and is intended as a "*drop-in replacement for the memory controller*." The ChampSim port satisfies this property exactly as specified.

3	0	7 / 7	✓
INDEPENDENT EVIDENCE POINTS	COHERENCE CALLS FOUND IN DCM	ON-CHIP CACHES CORRECTLY EXCLUDED	PROPERTY VERIFIED